Advanced Healthcare Analytics Dashboard

Technical Documentation

Table of Contents

1. System Overview
2. Architecture
3. Installation & Setup
4. Data Pipeline
5. Feature Engineering
6. Machine Learning Components
7. User Interface
8. API Reference
9. Configuration
10. Security Considerations
11. Maintenance & Monitoring
12. Troubleshooting

System Overview

Purpose

The Advanced Healthcare Analytics Dashboard is a comprehensive clinical decision support system designed for healthcare professionals. It provides real-time patient monitoring, predictive analytics, and risk assessment capabilities through an intuitive web interface.

Key Capabilities

- **Multi-dimensional Health Analytics**: Comprehensive monitoring across vital signs, activity, nutrition, sleep, and behavioral patterns

- **Predictive Risk Assessment**: AI-powered early warning system using machine learning models
- **Clinical Decision Support**: Evidence-based recommendations and risk stratification
- **Interactive Visualizations**: Real-time charts, trends, and clinical insights
- **Behavioral Integration**: Psychological factor analysis in health outcomes

Target Users

- Healthcare professionals (doctors, nurses, clinical staff)
- Care coordinators and case managers
- Health researchers and analysts
- Clinical administrators

Architecture

Technology Stack

- **Frontend**: Streamlit (Python web framework)
- **Data Processing**: Pandas, NumPy
- **Visualization**: Plotly (interactive charts)
- **Machine Learning**: Scikit-learn
- **Data Source**: Google Sheets CSV export

System Components

```
┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│                                Web Interface                                                 │
│                                (Streamlit)                                                   │
├──────────────────────────────────────────────────────────────────────────────────────────────┤
│  Clinical    │  Vital Signs  │  Activity    │  Risk       │
│  Overview    │  Analysis     │  Wellness    │  Assessment │
├──────────────────────────────────────────────────────────────────────────────────────────────┤
```

```
──────────┤
│              Data Processing Layer                  │
│       (Pandas, NumPy, Feature Engineering)          │
├──────────────────────────────────────────────────────────────────────────
──────────┤
│              Machine Learning Engine                │
│    (Scikit-learn: Random Forest, Gradient Boosting)   │
├──────────────────────────────────────────────────────────────────────────
──────────┤
│                Data Source                          │
│            (Google Sheets CSV)                      │
└──────────────────────────────────────────────────────────────────────────
──────────┘
```

Data Flow

1. **Data Ingestion**: Automated CSV import from Google Sheets
2. **Preprocessing**: Data cleaning, validation, and standardization
3. **Feature Engineering**: Advanced metric calculation and clinical scoring
4. **Model Training**: Real-time machine learning model updates
5. **Analytics**: Risk assessment and predictive analysis
6. **Visualization**: Interactive dashboard rendering

Installation & Setup

Prerequisites

```
Python 3.8+
pip package manager
Internet connection for data access
```

Required Dependencies

```
pip install streamlit pandas numpy plotly scikit-learn datetime warnings
```

Environment Setup

```
# Clone repository
git clone [repository-url]
cd healthcare-dashboard

# Create virtual environment
python -m venv healthcare_env
source healthcare_env/bin/activate  # Linux/Mac
# healthcare_env\Scripts\activate  # Windows

# Install dependencies
pip install -r requirements.txt
```

Launch Application

```
streamlit run healthcare_dashboard.py
```

Configuration

Update the `SHEET_URL` constant in the configuration section:

```
SHEET_URL = "https://docs.google.com/spreadsheets/d/YOUR_SHEET_ID/
export?format=csv"
```

Data Pipeline

Data Source Schema

The system expects the following columns in the source data:

Required Columns

- `patientId` (string): Unique patient identifier
- `observationStart` (datetime): Timestamp of observation
- `observationEnd` (datetime): End time of observation

Vital Signs

- `heartRate` (numeric): Heart rate in BPM
- `spo2` (numeric): Oxygen saturation percentage
- `temperature` (numeric): Body temperature in Fahrenheit
- `bloodPressure` (string): Format "systolic/diastolic" (e.g., "120/80")

Activity & Wellness

- `stepsTaken` (numeric): Daily step count
- `sleepHours` (numeric): Hours of sleep
- `exerciseMinutes` (numeric): Exercise duration
- `calorieIntake` (numeric): Calories consumed
- `waterIntakeMl` (numeric): Water intake in milliliters
- `bathroomVisits` (numeric): Number of bathroom visits
- `mealsSkipped` (numeric): Number of skipped meals

Clinical Data

- `state` (string): Health state classification
- `alerts` (string): Clinical alerts or notes
- `behaviourTags` (string): Behavioral observations
- `emotionTags` (string): Emotional state tags

Demographics

- `age` (numeric): Patient age
- `gender` (string): Patient gender

Data Preprocessing Pipeline

1. DateTime Processing

```
df["observationStart"] = pd.to_datetime(df["observationStart"], utc=True, e
rrors="coerce")
df['date'] = df['observationStart'].dt.date
```

```
df['hour'] = df['observationStart'].dt.hour
df['day_of_week'] = df['observationStart'].dt.day_name()
```

2. Vital Signs Cleaning

- Extracts numeric values from mixed data types
- Handles various input formats
- Applies range validation

3. Blood Pressure Processing

```
df['systolic'] = df['bloodPressure'].str.extract(r'(\d+)').astype(float)
df['diastolic'] = df['bloodPressure'].str.extract(r'/(\d+)').astype(float)
df['pulse_pressure'] = df['systolic'] - df['diastolic']
df['mean_arterial_pressure'] = df['diastolic'] + (df['pulse_pressure'] / 3)
```

4. Missing Value Strategy

- **Numeric Data**: Patient-specific median imputation, fallback to global median
- **Categorical Data**: Forward fill within patient groups
- **Critical Vitals**: Interpolation for short gaps

Feature Engineering

Clinical Risk Scores

Cardiopulmonary Risk

```
df['cardiopulmonary_risk'] = ((df['heartRate'] - 70) / 30) + ((95 - df['spo2']) / 5)
```

Age-Adjusted Heart Rate

```
df['age_adjusted_hr'] = df['heartRate'] / (220 - df['age'].fillna(65))
```

Hypertension Risk

```
df['hypertension_risk'] = ((df['systolic'] > 140) | (df['diastolic'] > 90)).astyp
e(int)
```

Activity Metrics

Activity Intensity

```
df['activity_intensity'] = df['stepsTaken'] / (df['exerciseMinutes'] + 1)
```

Hydration Ratio

```
df['hydration_ratio'] = df['waterIntakeMl'] / (df['calorieIntake'] + 1)
```

Sleep Quality Classification

```
df['sleep_quality'] = np.where(
    (df['sleepHours'] >= 7) & (df['sleepHours'] <= 9), 'optimal',
    np.where(df['sleepHours'] < 7, 'insufficient', 'excessive')
)
```

Behavioral Processing

```
df['has_behavioral_concerns'] = df['behaviourTags'].notna() & (df['behavio
urTags'] != '')
df['behavioral_severity'] = df['behaviourTags'].str.count(',') + 1
```

Daily Aggregation Rules

```
daily_agg_rules = {
    # Vital signs - average
    'heartRate': 'mean',
    'spo2': 'mean',
    'temperature': 'mean',
```

```
    # Activities - sum
    'stepsTaken': 'sum',
    'exerciseMinutes': 'sum',

    # Risk scores - max (worst case)
    'cardiopulmonary_risk': 'max',

    # Behavioral - flags
    'has_behavioral_concerns': 'max'
}
```

Machine Learning Components

Model Architecture

Feature Groups

```
feature_groups = {
    'vital_signs': ['heartRate', 'spo2', 'temperature', 'systolic', 'diastolic'],
    'cardiovascular': ['pulse_pressure', 'mean_arterial_pressure', 'hr_variabilit
y'],
    'activity': ['stepsTaken', 'exerciseMinutes', 'activity_intensity'],
    'nutrition': ['calorieIntake', 'waterIntakeMl', 'hydration_ratio', 'mealsSkipp
ed'],
    'sleep': ['sleepHours'],
    'clinical_risk': ['cardiopulmonary_risk'],
    'behavioral': ['has_behavioral_concerns', 'behavioral_severity'],
    'temporal': ['daily_observations'],
    'demographic': ['age']
}
```

Model Selection Process

1. **Feature Selection**: SelectKBest with f_classif scoring
2. **Model Candidates**:
    - Random Forest Classifier

- Gradient Boosting Classifier

3. **Selection Criteria**: Cross-validation accuracy
4. **Training Split**: 80% training, 20% validation

Model Training Pipeline

```
def train_advanced_health_predictor(daily_df):
    # Feature preparation
    X_features = daily_df[enhanced_features]
    y = daily_df['state']

    # Patient encoding
    encoder = OneHotEncoder(handle_unknown='ignore')
    patient_encoded = encoder.fit_transform(X[['patientId']])

    # Feature selection
    selector = SelectKBest(score_func=f_classif, k=min(25, X_combined.sha
pe[1]))
    X_selected = selector.fit_transform(X_combined, y)

    # Model comparison
    models = {
        'RandomForest': RandomForestClassifier(n_estimators=100, class_wei
ght='balanced'),
        'GradientBoosting': GradientBoostingClassifier(n_estimators=100)
    }

    # Best model selection
    return best_model, accuracy, features
```

Prediction Capabilities

- **Health State Classification**: Predicts patient stability/risk status
- **Feature Importance**: Identifies key health indicators
- **Trend Analysis**: Detects patterns in health metrics
- **Risk Scoring**: Quantitative risk assessment

User Interface

Dashboard Structure

Navigation Tabs

1. **Clinical Overview**: Key metrics and summary
2. **Vital Signs Analysis**: Comprehensive vital sign monitoring
3. **Activity & Wellness**: Physical activity and lifestyle metrics
4. **Behavioral Insights**: Psychological and behavioral patterns
5. **Risk Assessment**: Clinical risk stratification
6. **Predictive Analytics**: Forecasting and trend analysis

Patient Selection Interface

```
st.sidebar.header("Patient Selection & Filters")
patient_ids = sorted(df['patientId'].unique())
selected_patient = st.sidebar.selectbox("Select Patient:", patient_ids)
date_range = st.sidebar.date_input("Analysis Period:", ...)
```

Metric Display Components

```
# Key performance indicators
col1, col2, col3, col4, col5 = st.columns(5)
with col1:
    st.metric("Age", age)
with col2:
    st.metric("Current State", current_state, delta_color=state_color)
```

Visualization Components

Multi-Plot Vital Signs Dashboard

```
fig = make_subplots(
    rows=2, cols=2,
    subplot_titles=('Heart Rate Trend', 'SpO2 Levels', 'Temperature', 'Blood P
```

```
ressure')
)
```

Interactive Risk Assessment

```
fig_risk = px.bar(
    x=list(risk_scores.keys()),
    y=list(risk_scores.values()),
    color=list(risk_scores.values()),
    color_continuous_scale="Reds"
)
```

Trend Analysis with Prediction Lines

```
fig_trend = px.line(patient_daily, x='date', y=metric, markers=True)
# Add trend line
trend_line = coeffs[0] * x + coeffs[1]
fig_trend.add_scatter(y=trend_line, mode='lines', name='Trend')
```

API Reference

Core Functions

`load_and_preprocess_data()`

Purpose: Primary data loading and preprocessing function **Returns**: Tuple (raw_dataframe, daily_aggregated_dataframe)**Caching**: Streamlit @st.cache_data decorator

```
@st.cache_data
def load_and_preprocess_data():
    # Load from Google Sheets
    # Clean and process data
    # Create daily aggregations
    return df, daily_df
```

`create_daily_aggregation(df)`

Purpose: Aggregates hourly observations into daily summaries **Parameters**:

- `df`: Raw observation dataframe **Returns**: Daily aggregated dataframe

`train_advanced_health_predictor(daily_df)`

Purpose: Trains and selects best machine learning model **Parameters**:

- `daily_df`: Daily aggregated patient data **Returns**: Tuple (model_info_dict, accuracy_score, feature_list)

`generate_clinical_insights(patient_data, daily_data)`

Purpose: Generates clinical insights and recommendations **Parameters**:

- `patient_data`: Individual patient observations
- `daily_data`: Daily aggregated data for patient **Returns**: Insights dictionary with vital stability, activity patterns, risk factors

Configuration Constants

```
SHEET_URL = "https://docs.google.com/spreadsheets/d/.../export?format=
csv"

# Streamlit page configuration
st.set_page_config(
    page_title="Advanced Healthcare Dashboard",
    page_icon="",
    layout="wide"
)
```

Error Handling

Data Loading Errors

```
try:
    df = pd.read_csv(SHEET_URL)
except Exception as e:
    st.error(f"Error loading data: {str(e)}")
    return None, None
```

Model Training Errors

```
if len(all_features) < 5:
    st.warning("Insufficient features for advanced modeling")
    return None, None, None
```

Configuration

Environment Variables

```
# Optional: Set custom data source
export HEALTHCARE_DATA_URL="your_custom_url"

# Optional: Set cache timeout (seconds)
export STREAMLIT_CACHE_TTL=300
```

Streamlit Configuration

Create `.streamlit/config.toml`:

```
[server]
maxUploadSize = 50
enableCORS = false

[theme]
primaryColor = "#FF6B6B"
backgroundColor = "#FFFFFF"
secondaryBackgroundColor = "#F0F2F6"
textColor = "#262730"
font = "sans serif"
```

Clinical Thresholds

Modify thresholds in the configuration section:

```
CLINICAL_THRESHOLDS = {
    'heart_rate': {'min': 60, 'max': 100},
    'spo2': {'min': 95, 'max': 100},
```

```
    'temperature': {'min': 97.0, 'max': 99.5},
    'steps': {'min': 5000, 'target': 10000},
    'sleep': {'min': 7, 'max': 9}
}
```

Security Considerations

Data Privacy

- **Patient Identification**: All patient IDs should be anonymized
- **Data Transmission**: Ensure HTTPS for production deployment
- **Access Control**: Implement user authentication for clinical environments

HIPAA Compliance

- Store logs securely
- Implement audit trails
- Ensure data encryption at rest
- Regular security assessments

Access Patterns

```
# Example: Role-based access control
if user_role == 'physician':
    show_full_dashboard()
elif user_role == 'nurse':
    show_monitoring_dashboard()
elif user_role == 'admin':
    show_system_analytics()
```

Data Validation

```
def validate_patient_data(df):
    # Check for required columns
    required_cols = ['patientId', 'observationStart']
    missing_cols = [col for col in required_cols if col not in df.columns]
```

```
    if missing_cols:
        raise ValueError(f"Missing required columns: {missing_cols}")

    # Validate data ranges
    if 'heartRate' in df.columns:
        invalid_hr = df[(df['heartRate'] < 30) | (df['heartRate'] > 200)]
        if len(invalid_hr) > 0:
            st.warning(f"Found {len(invalid_hr)} invalid heart rate readings")
```

Maintenance & Monitoring

System Health Checks

Data Quality Monitoring

```
def check_data_quality(df):
    metrics = {
        'total_records': len(df),
        'unique_patients': df['patientId'].nunique(),
        'date_range': (df['observationStart'].min(), df['observationStart'].max
()),
        'completeness': (1 - df.isna().sum() / len(df)).mean()
    }
    return metrics
```

Model Performance Monitoring

```
def monitor_model_performance(model, test_data):
    predictions = model.predict(test_data)
    accuracy = accuracy_score(y_true, predictions)

    if accuracy < 0.7:
        st.warning("Model performance degradation detected")
        # Trigger retraining
```

Logging Configuration

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
    handlers=[
        logging.FileHandler('healthcare_dashboard.log'),
        logging.StreamHandler()
    ]
)
```

Backup Procedures

- **Data Backup**: Regular exports of processed data
- **Model Checkpoints**: Save trained models for quick recovery
- **Configuration Backup**: Version control for settings

Update Procedures

1. **Data Schema Updates**: Version migrations for new columns
2. **Model Updates**: A/B testing for new algorithms
3. **UI Updates**: User acceptance testing for interface changes

---

Troubleshooting

Common Issues

Data Loading Problems

Symptom: "Error loading data" message **Solutions**:

1. Check Google Sheets URL accessibility
2. Verify CSV export permissions
3. Check internet connectivity
4. Validate data format

```
# Debug data loading
try:
    response = requests.get(SHEET_URL)
    print(f"HTTP Status: {response.status_code}")
    print(f"Content Length: {len(response.content)}")
except Exception as e:
    print(f"Connection Error: {e}")
```

Model Training Failures

Symptom: "Insufficient features for modeling" **Solutions**:

1. Check data completeness
2. Verify column names match expected schema
3. Ensure minimum sample size (30+ records)

Performance Issues

Symptom: Slow dashboard loading **Solutions**:

1. Clear Streamlit cache: `st.cache_data.clear()`
2. Reduce date range for analysis
3. Optimize data preprocessing

Visualization Problems

Symptom: Charts not displaying **Solutions**:

1. Check Plotly installation
2. Verify data types for chart columns
3. Clear browser cache

Debug Mode

Enable debug logging:

```
import logging
logging.getLogger('streamlit').setLevel(logging.DEBUG)

# Add debug information
```

```
st.write("Debug Info:")
st.write(f"Data shape: {df.shape}")
st.write(f"Available columns: {list(df.columns)}")
st.write(f"Date range: {df['observationStart'].min()} to {df['observationStar
t'].max()}")
```

Performance Optimization

Caching Strategy

```
# Cache expensive operations
@st.cache_data(ttl=300)  # 5-minute cache
def expensive_calculation(data):
    return processed_data

# Clear cache when needed
if st.button("Refresh Data"):
    st.cache_data.clear()
    st.experimental_rerun()
```

Memory Management

```
# Efficient data loading
def load_data_chunked(url, chunk_size=10000):
    chunks = []
    for chunk in pd.read_csv(url, chunksize=chunk_size):
        chunks.append(process_chunk(chunk))
    return pd.concat(chunks, ignore_index=True)
```

Error Recovery

```
def safe_metric_calculation(data, metric_name):
    try:
        return calculate_metric(data)
    except Exception as e:
```

```
        st.error(f"Error calculating {metric_name}: {e}")
        return None
```

Deployment

Local Development

```
streamlit run healthcare_dashboard.py --server.port 8501
```

Production Deployment

Docker Container

```
FROM python:3.9-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt

COPY healthcare_dashboard.py .
EXPOSE 8501

CMD ["streamlit", "run", "healthcare_dashboard.py", "--server.address", "0.
0.0.0"]
```

Cloud Deployment

- **Streamlit Cloud**: Direct deployment from GitHub
- **AWS/GCP**: Container deployment with load balancing
- **Azure**: App Service deployment

Monitoring in Production

- **Health Checks**: Regular endpoint monitoring
- **Performance Metrics**: Response time tracking
- **Error Tracking**: Automated error reporting

- **Usage Analytics**: User interaction tracking

Support & Maintenance

Documentation Updates

- Update technical documentation with new features
- Maintain API documentation
- Update deployment guides

User Training

- Clinical staff onboarding
- Feature update training
- Best practices documentation

Support Channels

- Technical support ticket system
- User feedback collection
- Feature request tracking

This documentation serves as a comprehensive guide for healthcare professionals, developers, and system administrators working with the Advanced Healthcare Analytics Dashboard. Regular updates ensure accuracy with system evolution.

Version: 1.0

Last Updated: 2024

Maintenance Contact: [Technical Team]